

6-core-model

1 Core Model

The objective for this model is to predict the SD value for each nozzle given a set of parameters.

```
[ ]: import polars as pl
import numpy as np
import pandas as pd
from sklearn.preprocessing import LabelEncoder, OneHotEncoder
from sklearn.decomposition import PCA
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.multioutput import MultiOutputRegressor
from sklearn.model_selection import cross_val_score, train_test_split
from sklearn.metrics import mean_absolute_error
from sklearn.compose import ColumnTransformer
from sklearn.linear_model import Ridge
from sklearn.pipeline import Pipeline
import xgboost as xgb

PARQUET_PATH = "../../../data/interim/collapsed_waveform_tuning.parquet"

pd.set_option('display.max_rows', None)

df = pl.read_parquet(PARQUET_PATH)

feature_cols = ['Color$', 'HeadIdx$', 'V', 'F_r', 'dt2', 'Coverage$']
value_cols = [c for c in df.columns if c.startswith("Value_")]

X_raw = df.select(feature_cols).to_pandas()
y = df.select(value_cols).to_numpy()
```

2 Approach

2.0.1 Encoding

Feature	Current Form	Options	Recommendation
Color\$	String (C,M,Y,K)	Label encode / one-hot	Label encode — 4 balanced classes, ordinal doesn't matter for most models
HeadIdx#	Integer 1–30	As-is	Keep as integer — spatial adjacency gives ordering real physical meaning
V	Float (20–30)	As-is / MinMax	MinMax scale — evenly spaced, meaningful physical order
F_r	Float (1.02–1.379)	As-is / MinMax	MinMax scale — continuous physical dimension, model should interpolate
dt2	Float (-1100 to -300)	As-is / MinMax	MinMax scale — evenly spaced, meaningful order
Coverage#	Float (2–31)	As-is / MinMax	MinMax scale — evenly spaced, meaningful order

2.0.2 Scaling

Since all features are discrete and bounded, standard scaling options are:

- **MinMaxScaler** — maps everything to [0,1], good for distance-based methods and neural nets
- **StandardScaler** — zero mean, unit variance, good for PCA
- **No scaling** — fine for tree-based models

For dimensionality reduction, **StandardScaler before PCA** is the standard choice.

2.0.3 Dimensionality Reduction

With only 6 meaningful input features, classical PCA won't reduce much. The more interesting question is whether the **chip** × **waveform** × **color interactions** create separable structure in a lower-dimensional space. So the options worth trying are:

- **PCA** — quick baseline, tells you how much variance each component captures
- **t-SNE / UMAP** — better for visualising whether Color\$ classes are actually separable in the feature space

2.1 Encoding Categorical Values + Scaling Remaining Values

```
[ ]:
```

The value Colour is categorical and needs to be encoded. Since it consists of values without inherent order or ranking, we can use One-Hot Encoding.

```
[ ]: from sklearn.preprocessing import OneHotEncoder, MinMaxScaler
from sklearn.compose import ColumnTransformer

cat_onehot = ['Color$']
num_cols = ['V', 'F_r', 'dt2', 'Coverage#', 'HeadIdx#']

preprocessor = ColumnTransformer(
    transformers=[
        ("color", OneHotEncoder(handle_unknown="ignore", sparse_output=False),
        ↪cat_onehot),
        ("num", "passthrough", num_cols)
    ]
)

transformed = preprocessor.fit_transform(df[cat_onehot + num_cols].to_pandas())

# Reconstruct with readable column names
ohe_cols = preprocessor.named_transformers_['color'].
    ↪get_feature_names_out(cat_onehot).tolist()
feature_names = ohe_cols + num_cols

transformed_df = pd.DataFrame(transformed, columns=feature_names)
print(transformed_df.head(10))
print(f"\nShape: {transformed_df.shape}")
print(f"\nColumn ranges:")
print(transformed_df.describe().loc[['min', 'max']].round(4))
```

2.2 Dimensionality Reduction

2.2.1 With Scaling

```
[ ]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.manifold import TSNE

# SETUP
waveform_cols = [c for c in df.columns if 'Value' in c]
X = df[waveform_cols].to_numpy()
```

```

y = df['Color$'].to_numpy()

color_map = {'C': '#00AEEF', 'K': '#1a1a1a', 'M': '#E4007C', 'Y': '#FFF200'}
point_colors = [color_map[c] for c in y]

# SCALE BEFORE PCA
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 1. PCA
print("Running PCA...")
pca = PCA(n_components=10, random_state=42)
X_pca = pca.fit_transform(X_scaled)

explained = pca.explained_variance_ratio_
cumulative = np.cumsum(explained)

print(f"\nVariance explained per component:")
for i, (e, c) in enumerate(zip(explained, cumulative)):
    print(f" PC{i+1}: {e*100:.2f}% (cumulative: {c*100:.2f}%)")

fig, axes = plt.subplots(1, 3, figsize=(18, 5))
fig.suptitle("PCA on Waveform Features (1120 SD samples)", fontweight='bold',
    ↪ fontsize=13)

# Scree
axes[0].bar(range(1, 11), explained * 100, color='steelblue', edgecolor='white')
axes[0].plot(range(1, 11), cumulative * 100, 'o--', color='tomato',
    ↪ label='Cumulative')
axes[0].axhline(90, color='gray', linestyle='--', linewidth=0.8, label='90%
    ↪ threshold')
axes[0].set_title("Explained Variance", fontweight='bold')
axes[0].set_xlabel("Principal Component")
axes[0].set_ylabel("Variance Explained (%)")
axes[0].legend()

# PC1 vs PC2
for label in ['C', 'K', 'M', 'Y']:
    mask = y == label
    axes[1].scatter(X_pca[mask, 0], X_pca[mask, 1],
        c=color_map[label], label=label, alpha=0.3, s=5)
axes[1].set_title("PC1 vs PC2", fontweight='bold')
axes[1].set_xlabel(f"PC1 ({explained[0]*100:.1f}%)")
axes[1].set_ylabel(f"PC2 ({explained[1]*100:.1f}%)")
axes[1].legend(title='Color$', markerscale=3)

# PC2 vs PC3

```

```

for label in ['C', 'K', 'M', 'Y']:
    mask = y == label
    axes[2].scatter(X_pca[mask, 1], X_pca[mask, 2],
                    c=color_map[label], label=label, alpha=0.3, s=5)
axes[2].set_title("PC2 vs PC3", fontweight='bold')
axes[2].set_xlabel(f"PC2 ({explained[1]*100:.1f}%)"
axes[2].set_ylabel(f"PC3 ({explained[2]*100:.1f}%)"
axes[2].legend(title='Color$', markerscale=3)

plt.tight_layout()
plt.savefig("pca_waveform.png", dpi=150, bbox_inches='tight')
plt.show()

# 2. PCA LOADINGS - which waveform positions drive separation?
print("\nTop 5 waveform positions driving PC1:")
pc1_loadings = pd.Series(np.abs(pca.components_[0]), index=waveform_cols).
    ↪sort_values(ascending=False)
print(pc1_loadings.head(5))

print("\nTop 5 waveform positions driving PC2:")
pc2_loadings = pd.Series(np.abs(pca.components_[1]), index=waveform_cols).
    ↪sort_values(ascending=False)
print(pc2_loadings.head(5))

# 3. t-SNE on PCA-reduced data
# Run t-SNE on first 10 PCs rather than raw 1120 dims - faster, less noisy
print("\nRunning t-SNE on top 10 PCs...")
sample_size = 15000
idx = np.random.choice(len(X_pca), sample_size, replace=False)
X_tsne_input = X_pca[idx, :10]
y_sample = y[idx]

tsne = TSNE(n_components=2, perplexity=50, random_state=42, n_jobs=-1)
X_tsne = tsne.fit_transform(X_tsne_input)

fig, ax = plt.subplots(figsize=(10, 8))
for label in ['C', 'K', 'M', 'Y']:
    mask = y_sample == label
    ax.scatter(X_tsne[mask, 0], X_tsne[mask, 1],
                c=color_map[label], label=label, alpha=0.4, s=8)
ax.set_title("t-SNE on Waveform SD Samples - coloured by Color$",
    ↪fontweight='bold', fontsize=13)
ax.set_xlabel("t-SNE 1")
ax.set_ylabel("t-SNE 2")
ax.legend(title='Color$', markerscale=3, fontsize=10)
plt.tight_layout()
plt.savefig("tsne_waveform.png", dpi=150, bbox_inches='tight')

```

```
plt.show()
```

2.2.2 Without Scaling

```
[ ]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE

# SETUP
waveform_cols = [c for c in df.columns if 'Value' in c]
X = df[waveform_cols].to_numpy()
y = df['Color$'].to_numpy()

color_map = {'C': '#00AEEF', 'K': '#1a1a1a', 'M': '#E4007C', 'Y': '#FFF200'}

# 1. PCA WITHOUT SCALING
print("Running PCA without scaling...")

pca = PCA(n_components=10, random_state=42)
X_pca = pca.fit_transform(X)

explained = pca.explained_variance_ratio_
cumulative = np.cumsum(explained)

print(f"\nVariance explained per component:")
for i, (e, c) in enumerate(zip(explained, cumulative)):
    print(f"  PC{i+1}: {e*100:.2f}% (cumulative: {c*100:.2f}%)")

fig, axes = plt.subplots(1, 3, figsize=(18, 5))
fig.suptitle("PCA on Raw Waveform Features", fontweight='bold', fontsize=13)

# Scree plot
axes[0].bar(range(1, 11), explained * 100,
            color='steelblue', edgecolor='white')

axes[0].plot(range(1, 11), cumulative * 100,
            'o--', color='tomato', label='Cumulative')

axes[0].axhline(90, color='gray', linestyle='--',
               linewidth=0.8, label='90% threshold')

axes[0].set_title("Explained Variance", fontweight='bold')
axes[0].set_xlabel("Principal Component")
axes[0].set_ylabel("Variance Explained (%)")
axes[0].legend()
```

```

# PC1 vs PC2
for label in ['C', 'K', 'M', 'Y']:
    mask = y == label

    axes[1].scatter(
        X_pca[mask, 0],
        X_pca[mask, 1],
        c=color_map[label],
        label=label,
        alpha=0.3,
        s=5
    )

axes[1].set_title("PC1 vs PC2", fontweight='bold')
axes[1].set_xlabel(f"PC1 ({explained[0]*100:.1f}%)")
axes[1].set_ylabel(f"PC2 ({explained[1]*100:.1f}%)")
axes[1].legend(title='Color$', markerscale=3)

# PC2 vs PC3
for label in ['C', 'K', 'M', 'Y']:
    mask = y == label

    axes[2].scatter(
        X_pca[mask, 1],
        X_pca[mask, 2],
        c=color_map[label],
        label=label,
        alpha=0.3,
        s=5
    )

axes[2].set_title("PC2 vs PC3", fontweight='bold')
axes[2].set_xlabel(f"PC2 ({explained[1]*100:.1f}%)")
axes[2].set_ylabel(f"PC3 ({explained[2]*100:.1f}%)")
axes[2].legend(title='Color$', markerscale=3)

plt.tight_layout()
plt.savefig("pca_waveform_raw.png", dpi=150, bbox_inches='tight')
plt.show()

# 2. PCA LOADINGS
print("\nTop 5 waveform positions driving PC1:")

pc1_loadings = (
    pd.Series(np.abs(pca.components_[0]), index=waveform_cols)
    .sort_values(ascending=False)

```

```

)

print(pc1_loadings.head(5))

print("\nTop 5 waveform positions driving PC2:")

pc2_loadings = (
    pd.Series(np.abs(pca.components_[1]), index=waveform_cols)
    .sort_values(ascending=False)
)

print(pc2_loadings.head(5))

# 3. t-SNE on PCA-reduced data
print("\nRunning t-SNE on top 10 PCs...")

sample_size = 15000
idx = np.random.choice(len(X_pca), sample_size, replace=False)

X_tsne_input = X_pca[idx, :10]
y_sample = y[idx]

tsne = TSNE(
    n_components=2,
    perplexity=50,
    random_state=42,
    n_jobs=-1
)

X_tsne = tsne.fit_transform(X_tsne_input)

fig, ax = plt.subplots(figsize=(10, 8))

for label in ['C', 'K', 'M', 'Y']:
    mask = y_sample == label

    ax.scatter(
        X_tsne[mask, 0],
        X_tsne[mask, 1],
        c=color_map[label],
        label=label,
        alpha=0.4,
        s=8
    )

ax.set_title(
    "t-SNE on Raw Waveform Features",

```



```

        fontweight='bold',
        fontsize=13
    )

    ax.set_xlabel("t-SNE 1")
    ax.set_ylabel("t-SNE 2")

    ax.legend(title='Color$', markerscale=3, fontsize=10)

    plt.tight_layout()
    plt.savefig("tsne_waveform_raw.png", dpi=150, bbox_inches='tight')
    plt.show()

```

```
[ ]: X_diff = np.diff(X, axis=1)
```

2.2.3 PCA

The first dimensionality reduction we will try is Principal Component Analysis.

```
[ ]: X_processed = preprocessor.fit_transform(X_raw)

X_dense = X_processed.toarray() if hasattr(X_processed, "toarray") else
↳X_processed

```

```
[ ]: pca = PCA(n_components=0.95) # keep 95% variance
X_pca = pca.fit_transform(X_dense)

```

Evaluation

```
[ ]: print("Components used:", pca.n_components_)
print("Explained variance ratio (first 10):", pca.explained_variance_ratio_[:
↳10])
print("Total explained variance:", pca.explained_variance_ratio_.sum())

head_start = len(cat_ordinal)
head_end = head_start + len(preprocessor.named_transformers_["head"].
↳categories_[0])

print("HeadIdx one-hot dimensionality:", len(preprocessor.
↳named_transformers_["head"].categories_[0]))

loadings = np.abs(pca.components_)
head_contribution = loadings[:, head_start:head_end].sum()

print("Relative HeadIdx contribution:", head_contribution)

```

These results show that there is one component dominating the dataset, meaning all variation lies along a single axis.

HeadIdx is not the driving variance, which means there's another dominating feature (most likely dt2, from previous analysis).

2.3 Identifying First Component

```
[ ]: pc1 = pca.components_[0]

feature_names = preprocessor.get_feature_names_out()

top_idx = np.argsort(np.abs(pc1))[:, :-1][:10]

for i in top_idx:
    print(feature_names[i], pc1[i])
```

With this, we confirm **dt2 is the dominant axis**, since it has the most influence in the results.

2.4 dt2 in PCA

Since dt2 is closer to a calibration parameter rather than a feature in the usual sense, the current approach will not be accurate if we don't treat it differently.

First we need to **normalise out dt2 levels** in order to see how the system looks like after timing alignment.

```
[ ]: from sklearn.linear_model import LinearRegression
import numpy as np

dt2 = X_raw[['dt2']].values

X_other = X_raw[['V', 'F_r', 'Coverage#']]
```

```
[ ]: model = LinearRegression()
model.fit(dt2, X_other)

X_pred = model.predict(dt2)
residuals = X_other - X_pred
```

```
[ ]: X_aligned = pd.DataFrame(residuals, columns=X_other.columns)
X_aligned['Color$'] = X_raw['Color$']
X_aligned['HeadIdx#'] = X_raw['HeadIdx#']
```

```
[ ]: X_proc = preprocessor.fit_transform(X_aligned)
X_dense = X_proc.toarray() if hasattr(X_proc, "toarray") else X_proc

pca = PCA()
X_pca = pca.fit_transform(X_dense)

print(pca.explained_variance_ratio_[:10])
```

```
[ ]: import numpy as np

print("New explained variance:", np.cumsum(pca.explained_variance_ratio_)[:10])
```

3 PCA is not the solution!

let's keep working in [another notebook](#)